A02 ·이분법과 실패를 보존하는 근 찾기

과제 큐시트

선행: A01　　**독립 작업:** 3–5 시간 (계획치)

작은 잔차는 근에 가까워졌다는 뜻일까?

같은 함수 전체를 작은 양의 상수로 곱한다. 근의 위치, 잔차의 크기, 이분법의 구간 폭 중 무엇이 변하는지 계산 전에 나누어 적자.

먼저 한두 문장으로 답한 뒤 아래 설명을 읽자. 첫 예측이 틀렸거나 아직 모르겠다고 적는 것은 감점하지 않는다.

필요한 관계와 읽을 위치

$F(a)F(b) < 0$ 는 연속성 가정 아래에서 근 존재의 충분조건이다. 실제 부호 비교는 곱셈 없이 한다. $b_k - a_k = (b_0 - a_0)2^{-k}$.

- `notes/chapters/ch02_numerics.md`
- `learner/src/numerics/ns_bisection.f90`

계산으로 옮기기

기본 경로는 제공 함수의 호출과 해석이다. 아래 `learner` 시험은 같은 물리 함수를 직접 작성한 경우에 실행한다. 제공 예제를 검사할 때에는 명시적으로 `calculation` 을 선택한다.

1. 풀이에서 bracket 의 연속성 전제와 갱신할 절반을 확인한다.
2. 제공 근 찾기를 호출하고 정상 근·끝점 근·실패 상태를 비교한다.
3. 함수의 배율을 바꿔 잔차 조건과 좌표 조건을 따로 읽는다. 직접 구현은 같은 시험을 사용하는 선택 경로다.

실행해서 무엇을 볼 것인가

패키지 루트에서 실행한다. `--plan` 은 경로만 확인하며 계산 성공을 뜻하지 않는다.

```
python tools/run_assignment.py A02 --plan
python tools/run_assignment.py A02 --run \
  --workspace learner --output ../A02_result.json
```

결과를 설명하기

근은 함수의 전체 배율에 무관하다. 잔차와 좌표 오차를 같은 조건으로 취급했다면, 어느 종료 기준을 썼는지 먼저 찾아 비교하자.

남길 증거. bracket 폭과 반복 횟수 표; 성공 사례와 실패 상태 한 개; 함수 스케일 변화의 해석

판단 기준. 2 점 예: F 스케일은 root 를 바꾸지 않지만 residual stopping 은 달라짐을 실행으로 설명한다. 최초 예측의 정오보다, 결과를 본 뒤 이유를 고치고 다른 입력에도 적용하는지를 본다.

막혔을 때

힌트 1. 매 반복마다 유지되어야 하는 조건을 assert 대신 종이에 먼저 적는다.

힌트 2. 아주 큰 반대 부호 끝점에서 중점식의 overflow 가능성을 검사한다.

다음 만남에 다시 묻기

다음 회차에는 연속인 삼차함수와 pole 을 가진 함수에서 같은 부호 반전이 무엇을 뜻하는지 설명한다. 함수값을 보기 전에 필요한 전제를 말한다. 먼저 짧게 답한 뒤 본문과 자신의 기록을 열어 확인한다.

범위. pole 및 discontinuity 에서는 부호변화만으로 근을 주장할 수 없다.